

Java 课程作业 2026

人工智能辅助信息 王者荣耀管理系统

Java面向对象编程 | 负责任的AI使用 | Git流程证据

总分	20 分
提交	Java 源代码、文档、AI 证据、Git 历史记录和测试证据。学生可以使用 AI 工具,但他们必须记录提示、展示人类
核心思想	判断、保留 Git 证据并解释 Java 设计决策。
受到推崇的模式	首先是控制台应用程序;只有在核心需求稳定后才会考虑图形用户界面或高级功能。

设计理念

这项重新设计的作业将人工智能的使用视为一项需要评估的学术实践,而非捷径。学生必须展现出编程能力和人工智能素养:包括规划、提示、验证、调试、反思和责任承担。

内容

1. 教学原理	3
2 课程概述	3
2.1 项目名称.....	3
2.2 任务概述.....	3
2.3 学习成果。	4
3. Java 核心设计要求	4
3.1 必修课程。	4
3.2 必备的Java概念.....	4
4. 初始数据集要求	5
5. 功能需求	5
5.1 球员查询...	5
5.2 团队概览。	6
5.3 英雄详情.....	6
5.4 装备统计.....	6
5.5 比赛历史...	6
5.6 排行榜.....	7
5.7 数据管理。	7
5.8 身份验证...	7
6 人工智能使用要求和证据6.1 人工智能文件夹要求。	7
6.2 提示记录： prompts.md 。	8
6.3 多智能体证据： agent-log.md 。	9
6.4 反思： reflection.md 。	9
7. 必需的 Git 证据	10
7.1 最低 Git 要求。	10
8 必需的计划.md	11
8.1 建议时间表。	11
9. 源代码和测试要求	11
9.1 源代码要求。	11
9.2 测试要求.....	12

10项额外加分功能	13
10.1 战斗模拟。	13
10.2 推荐引擎。	13
10.3 图形用户界面...	13
10.4 数据持久性。	13
10.5 高级人工智能反思...	14
11. 提交要求	14
12 学术诚信与人工智能政策	14
13 评分标准	15
13.1 等级带.....	16
14. 最低合格清单	16
附录 A:学生工作流程	17
附录 B:快速质量指南	17
附录 C:推荐的 README.md 结构	17
参考	18

1. 教学原理

本次课程作业重新设计保留了原有的《王者荣耀》信息管理系统主题及其Java面向对象编程基础,但增加了一个结构化的AI应用层。修订后的作业旨在同时评估两项能力:

- 学生是否理解 Java 编程、面向对象建模、集合、文件 I/O、异常处理和测试;
- 学生能否在学术环境中负责任、透明且批判性地使用人工智能工具编程任务。

该设计借鉴了斯坦福大学的三项教学理念。首先,斯坦福大学 CS106B 课程的 AI 编程部分要求学生使用逻辑学习模型 (LLM),然后报告他们的模型选择、提示、集成过程、调试和反思[1]。其次,斯坦福大学教学共享平台建议在课程材料中明确阐述 AI 政策,包括允许的行为、必须披露的信息以及该政策如何维护学术诚信[2]。第三,斯坦福大学 SCALE 项目的“提示问题”框架将部分编程教育的重点从编写代码转移到构建提示、阅读生成的代码和评估其正确性[3]。

评估原则

允许使用人工智能辅助,但不允许使用隐藏式人工智能辅助。学生需提供有用的提示、清晰的决策过程、良好的 Git 代码证据、准确的反思以及经证实的 Java 理解能力,方可获得学分。

2 课程概述

2.1 项目名称

《王者荣耀》人工智能辅助信息管理系统

2.2 任务概要

学生必须设计并实现一个Java系统,用于管理《王者荣耀》游戏中的玩家、英雄、装备、队伍和比赛记录。该系统应支持搜索、报告、数据管理、身份验证、排行榜以及可选的高级功能。

学生应以受控且有记录的方式使用人工智能工具。最终提交的作品必须包含以下内容:

- 一个可运行的 Java 应用程序;
- 清晰的面向对象设计;
- 一份详细的计划.md;
- 实际的人工智能提示和多智能体交互记录;
- Git 历史记录显示了迭代开发过程;
- 检验证据并进行最终反思。

2.3 学习成果

完成本课程后,学生应能够:

- 1. 使用类、继承、接口、集合和封装设计一个 Java 程序。
tion;
- 2. 实现菜单驱动型应用程序,具备搜索、排名、数据管理和身份验证功能;
- 3. 使用文件 I/O 或其他持久化方法保存和加载结构化数据;
- 4. 使用 Git 记录迭代开发过程;
- 5. 负责任地使用人工智能工具,包括记录提示、评估输出和解释决策。
sions;
- 6. 对 AI 辅助生成的代码进行测试和调试,而不是盲目信任生成的代码。

3. Java 核心设计要求

3.1 必修课程

项目必须至少包含以下类:

班级	必要目的
人	系统用户的抽象超类。
玩家	Person类的子类;代表游戏玩家。
行政	Person的子类;表示具有数据管理权限的用户。
英雄	代表一个可玩英雄,包括类型、属性和装备兼容性。
设备	代表英雄可以装备的物品。
团队	代表一个包含多名球员的团队。
比赛记录	表示比赛结果、参赛者、英雄选择和日期。

学生可以根据需要选修额外课程,例如:

皮肤、符文、技能、游戏数据管理器、身份验证服务
搜索服务、排名服务、文件存储服务
推荐引擎、战斗模拟器、输入辅助程序

3.2 必要的Java概念

实施方案必须体现以下概念的有效运用:

概念	预期证据
遗产	玩家和管理员扩展了Person 的功能。
协会	一名玩家拥有多个英雄物品;一名英雄可以使用多个设备对象。

聚集体组成	或者 一个团队包含多个玩家对象。
界面	使用至少一个有意义的界面,例如可搜索的界面。 可报告、可持久化或可验证。
封装	字段应设为私有;访问权限应通过以下方式控制: 适当的方法。
多态性	在适当的地方使用通用超类或接口引用,例如将用户存储为Person 类型。
收藏	使用ArrayList、HashMap、Set、TreeMap或类似的集合。
异常处理	处理无效输入、缺失记录、重复 ID 和文件错误 加载错误。
文件 I/O	从文本、CSV、JSON 或其他格式保存和加载系统数据 文档格式。
枚举	使用枚举类型,例如HeroType、MatchResult、Role等。 设备类型。

重要的

如果一个程序将几乎所有逻辑都放在一个大型的Main类中,那么它的 Java 设计水平将会很低。即使某些菜单功能可以正常使用,也只能获得分数。

4. 初始数据集要求

初始数据集必须至少包含：

数据类型	最低要求
团队	3支队伍,每队至少5名队员。
球员	10 名玩家,每人至少拥有 3 名英雄。
英雄	15 位英雄,每位英雄至少可以装备 2 件装备。
设备	20件设备。
比赛记录	10场比赛记录。

数据集可能在早期开发阶段被硬编码。为了获得满分,系统应该支持通过文件保存和加载数据。

5. 功能需求

系统必须提供菜单驱动界面。控制台界面也是可以接受的。图形用户界面 (GUI)也行。
使用 Swing 或 JavaFX 的技能可以作为加分项提交。

5.1 球员查询

按ID或姓名搜索玩家。显示：

- 玩家ID和姓名；
- 团队；

- 等级和胜率；
- 拥有英雄；
- 每位英雄的装备物品。

5.2 团队概览

按 ID 或名称搜索球队。显示：

- 团队名称；
- 所有成员；
- 平均水平；
- 总比赛场次；
- 胜率；
- 队内最佳球员。

5.3 英雄详情

按姓名搜索英雄。显示：

- 英雄名称和英雄类型；
- 基础统计数据；
- 可用的或兼容的设备；
- 拥有该英雄的玩家；
- 如果实施,建议配备以下设备。

5.4 装备统计

至少根据一项指标对装备进行排名,例如：

- 使用次数；
- 平均评分；
- 使用该技能的英雄数量；
- 胜率贡献；
- 自定义分数。

学生必须解释文档中的排名公式。

5.5 比赛历史

获取球员或球队的最近N场比赛记录。显示：

- 对手；
- 日期；
- 结果；
- 英雄已选定；

- 胜负记录；
- 英雄选择率。

5.6 排行榜

按以下方式显示排名前X 位的球员：

- 胜率；
- 等级；
- 比赛场次；
- 自定义分数。

学生必须解释如何处理平局。

5.7 数据管理

管理员用户可以添加、删除和编辑：

- 球员；· 英雄；
- 设备；
- 团队；
- 比赛记录。

玩家用户可以：

- 查看自己的信息；
- 编辑有限的个人信息； · 查看他们的英雄和比赛历史记录； · 查看公开的英雄、队伍和排行榜信息。

5.8 身份验证

实现一个简单的登录和注销系统,至少包含两个角色：

- 行政；
- 玩家。

管理员用户可以创建、修改或删除所有数据。玩家用户只能查看常规数据并编辑自己的基本信息。

6. 人工智能使用要求及证据

允许并鼓励使用人工智能工具。学生可以使用 ChatGPT、Claude、Gemini、Copilot、Cursor、Codex 或其他人工智能编程助手等工具。但是，学生必须对提交的每一行代码负责。

即使程序能够运行,提交学生无法解释的AI生成代码也会得到低分。

6.1 必需的 AI 文件夹

请提交一个包含以下文件的ai/文件夹：

```
ai/提示.md 代理日志.md 反射.md
```

6.2 提示记录： prompts.md

学生必须记录项目过程中使用的所有重要提示。每条提示记录必须包含：

- 日期和时间；
- 使用的人工智能工具或模型；
- 代理人角色；
- 实际提示；
- 人工智能响应的简要总结；
- 该回复是被接受、修改还是拒绝；
- 相关 Git 提交哈希值。

列表 1:示例提示记录

```
## 提示 03

时间:2026年5月12日 21:30
工具/模型:ChatGPT / GPT-5.5 代理角色:Java架构师
相关提交:a13f91c

### 我的提示

我正在为“国王荣耀”信息系统设计一个Java面向对象编程课程项目。请建议一种使用继承、接口和集合的类结构。暂时不要编写完整的代码,重点放在类的职责和关系上。

### 人工智能响应摘要

AI 建议了人员、玩家、管理员、英雄、装备、队伍和比赛记录等选项。它还建议了可搜索和可持久化的接口。

### 我的决定

接受了总体结构,但将 Match 改为 MatchRecord,以避免命名歧义,并使类的职责更加清晰。
```

必需的

仅仅写“我使用了ChatGPT”是不够的。学生必须展示实际的提示信息,并解释他们如何处理AI的回复。

6.3 多智能体证据： agent-log.md

学生在开发过程中必须使用至少三种不同的AI代理角色。这些角色不必是不同的AI工具,也可以是分配给同一个模型的不同角色。

所需职位：

- 架构师代理**:类设计、UML 建议、模块规划；
- 实现代理**:选定的 Java 方法或类实现；
- 测试/审查代理**:错误查找、测试用例、代码审查。

可选角色包括重构代理、文档代理、安全代理和性能代理。

列表 2 :示例 agent-log.md 条目

```
# 代理日志

## 建筑师代理

主要贡献：
提出了初始的面向对象编程结构和关系。

人为决策:我接受了抽象的
Person 类,但拒绝了建议的 GameAccount 类,因为它对于所需的功能来说是不必要的。

相关提交： - 1a2b3c4 初始类
设计 - 2b3c4d5 添加 Person、Player、Admin

## 测试代理

主要贡献:发现删除英雄并不会
将其从玩家拥有的英雄列表中移除。

人为决策:我通过更新
GameDataManager 中的 deleteHero 函数手动修复了这个错误。

相关提交：
- 9f8e7d6 修复英雄删除一致性
```

6.4 反思： reflection.md

学生必须回答以下问题：

1. 您使用了哪些人工智能工具或模型？
2. 哪个提示最有用?为什么？
3. 人工智能生成的哪个建议是错误的、不完整的或具有误导性的？
4. 你如何检验人工智能生成的代码是否正确？
5. 你亲自修复了哪些漏洞,而不是让 AI 来修复？
6. 使用人工智能后,你对 Java 的哪个概念理解得更透彻了？
7. 你对 Java 的哪些概念仍然不确定？

8. 人工智能是让项目变得更容易、更难,还是两者兼而有之?请解释。
9. 最终项目的哪些部分主要由你撰写?
10. 哪些部分主要由人工智能生成或大量辅助?

7. 必需的 Git 证据

学生必须在整个项目过程中使用 Git。提交的代码仓库必须包含有意义的提交历史记录。只有一个最终提交是不可接受的。

7.1 最低 Git 要求

- 至少 12 次有意义的提交。
- 至少有 4 次提交必须是明显由人编写的,用于规划、调试或重构。提交。
- 至少 3 个提交必须与Architect Agent 关联。
- 至少 3 个提交必须链接到实施代理。
- 至少 2 个提交必须与测试/审查代理关联。

每条提交信息都应该使用以下前缀之一:

[人类]
[AI架构师]
【人工智能实施】
【AI评论】
[AI重构]
[文档]
[测试]
[使固定]

提交信息示例:

[人工] 创建初始项目结构和 README 文件 [AI架构师] 为“人物英雄团队”草拟面向对象编程类结构 [人工] 根据 UML 反馈修改类设计 [AI 实现] 实现玩家查找菜单 [测试] 为团队概览添加手动测试用例 [AI 审核] 审核数据删除的一致性 [修复] 修复审核过程中发现的英雄删除错误 [文档] 更新 plan.md 和 reflection.md 文件

学生必须提交以下输出结果:

```
git log --oneline --graph --decorate --all
```

在名为 git-history.txt 的文件中。

标记笔记

Git 证据的评估侧重于开发过程,而不仅仅是提交数量。冗长而无意义的提交列表远不如一个简短但清晰的流程 包括计划、实施、审查、修复和文档编写 来得有效。

8 必需的计划.md

学生必须提交一份精心撰写的plan.md 文件。该文件是课程作业的重要组成部分。

plan.md 的必要结构

- 1.项目目标 :解释系统的功能和用户群体。
- 2.需求分析 :列出所有必需的功能,并解释系统将如何实现这些功能。
逐一实施。
- 3. Java概念的应用 :解释继承、接口、多态、集合等概念的应用场景。
将使用 tions、异常处理、文件 I/O 和枚举。
- 4.课程设计 :描述每个主要课程及其职责。
- 5. UML 草稿:包含 UML 图图像或基于文本的 UML 描述。
- 6.数据设计 :解释初始数据集以及数据的存储方式。
- 7. AI 使用计划:说明将使用哪些 AI 代理以及每个代理的权限。
帮忙。
- 8.提示策略:解释提示将如何编写以及人工智能的输出将如何生成。
已确认。
- 9.开发时间表 :将项目分解为多个阶段。
- 10.测试计划 :列出每个主要功能的测试用例。
- 11.风险分析 :解释可能存在的风险和缓解策略。
- 12.最终反思占位符 :在实施后留出空间进行最终反思。

一份只写着“我将使用人工智能辅助我编程”的薄弱的plan.md文件会得到低分。一份强有力的 plan.md 文件则会得到低分。
plan.md 文件应该表明学生在编写代码之前已经制定了项目计划。

8.1 建议时间表

阶段	预期工作
第一阶段	阅读需求,创建代码仓库,编写第一个plan.md 文件。
第二阶段	向建筑师代理征求设计反馈意见;手动修改类结构。
第三阶段	实现模型类和初始数据。
第四阶段	实现菜单系统和搜索功能。
第五阶段	实现身份验证和管理员/玩家权限。
第六阶段	实现持久化和排名功能。
第七阶段	使用测试/审核代理查找错误;修复并记录决定。
第八阶段	完成文档编写、反思、Git导出和最终测试。

9. 源代码和测试要求

9.1 源代码要求

Java 项目必须：

- 编译成功；· 从清晰的主方法

运行；

- 具有清晰的包或文件夹结构；· 使用有意义的类名、方法

名和变量名；

- 避免将所有代码放在一个巨大的类中；
- 避免不必要的重复代码；
- 妥善处理无效的用户输入；
- 为重要逻辑添加注释；
- 能够被人类标记员理解。

建议结构：

```
src/  
  Main.java  
  model/  
    Person.java  
    Player.java  
    Admin.java  
    Hero.java  
    Equipment.java  
    Team.java  
    MatchRecord.java service/  
  
  GameDataManager.java  
    AuthenticationService.java SearchService.java  
    RankingService.java  
    FileStorageService.java util/  
    InputHelper.java DataInitializer.java  
docs/  
  plan.md design.mduml.png  
  test-cases.md ai/ prompts.md  
agent-  
  log.md  
  reflection.md  
  git-history.txt  
  README.md
```

9.2 测试要求

学生必须提交自动化测试或手动测试文档。至少应提交以下内容：

```
docs/test-cases.md
```

测试文档必须包含至少 10 个测试用例。每个测试用例应包含：

- 测试 ID；

- 功能已测试；
- 输入；
- 预期输出；
- 实际输出；
- 通过/不通过结果；
- 是否发现错误？

清单 3:示例测试用例

测试 04:按姓名查找玩家

输入:

搜索玩家名称:“李白”

预期结果:系统

将显示李白的队伍、等级、拥有的英雄和装备。

实际情况:

系统正确显示了玩家信息。

结果:

经过

编写有意义的 JUnit 测试可获得额外加分。

10项额外加分功能

学生可以通过实现以下一项或多项功能来获得额外学分。

10.1 战斗模拟

实现一个回合制战斗模拟器,使用英雄属性、装备和随机因素。它应该包括伤害计算、暴击或闪避、胜负结果和战斗报告。

10.2 推荐引擎

根据英雄类型、玩家偏好、胜率、装备使用情况或队伍配置推荐英雄或装备。推荐公式必须在文档中加以说明。

10.3 图形用户界面

使用 Swing 或 JavaFX 实现图形用户界面 (GUI)。它至少应支持玩家查询、队伍概览、英雄详情和排行榜。

10.4 数据持久性

支持使用文本文件、CSV 文件、JSON 文件或 JDBC 数据库保存和加载数据。仅当有明确的文档说明时才允许使用外部库。

10.5 高级人工智能反射

比较两个不同的AI模型或两种不同的代理角色解决同一问题的情况。例如,分别询问实现代理和审核代理如何实现排行榜排序,然后比较它们的正确性、可读性、缺陷和学习价值。

11. 提交要求

提交一个包含以下内容的.zip文件或 Git 仓库链接:

```
src/  
docs/  
ai/  
README.md  
plan.md git-  
history.txt
```

提交材料必须包含:

- 完整的Java源代码;
- plan.md;
- 设计文档或design.md;
- UML类图;
- prompts.md;
- agent-log.md;
- reflection.md;
- git-history.txt;
- 测试文档; · 程序运行说

明。

请勿仅提交屏幕截图。请勿仅提交 AI 聊天记录而不提供源代码。请勿提交您无法解释的代码。

12 学术诚信与人工智能政策

本课程作业允许使用人工智能。但是,以下行为是不允许的:

- 提交自己不理解的AI生成的代码;
- 删除或隐藏人工智能提示;
- 伪造 Git 历史记录;
- 将人工智能生成的作品据为己有;
- 利用人工智能伪造测试结果;
- 使用人工智能进行虚假写作;
- 提交其他学生的代码。

学生可将人工智能用于:

- 了解需求；
- 集思广益设计；
- 生成小型代码示例；
- 调试；
- 重构；
- 编写测试用例；
- 改进文档。

学生必须亲自核实：

- 代码可以编译；
- 代码运行；
- 代码符合要求；
- 正确运用了Java概念；
- 该文档准确地描述了最终系统。

13 评分标准

类别	分数	标准
Java 设计与理解	5	优秀的提交作品展现了清晰的面向对象编程设计、继承、接口、集合、封装、异常处理、文件I/O以及可解释的设计决策。较弱的提交作品大多是过程式的或
功能完备性	4	无法解释。 满分要求完成工作查找、团队概述等。 英雄详情、装备统计、比赛记录 排行榜、数据管理和身份验证。
人工智能使用证据4		满分要求提供实际的提示记录、多代理工作流程、深入的思考,以及对已接受、已修改和已拒绝的 AI 建议的清晰解释。
Git 流程证据	3	满分要求清晰的迭代式 Git 历史记录,展现人工规划和 AI 辅助实施。
plan.md和文档	2	审查、修复、测试和文档编写。 满分需要详细的规划、UML图和设计。 说明、测试计划、实施说明 以及准确的最终文件。
测试与可靠性	1	满分需要清晰的手动测试用例或带有调试证据的自动测试用例。
额外加分或创造力	1	表彰具有实际意义的额外功能,例如图形用户界面 (GUI)。 持久性、推荐引擎、战斗模拟或高级人工智能反思。

13.1 等级带

年级	分数	描述
A	16-20	核心功能运行良好,Java 设计强大,AI 和 Git 的证据也充分。 内容详尽,文档专业,并且至少有一项额外内容 功能已实现。
B	10-15	大多数核心功能都能正常工作,Java 结构尚可接受,人工智能方面的证据和文档虽然存 在,但并不完善。
C	6-9	部分必要功能尚未实现,Java 设计不够精细。 文件或人工智能证据不足。
F	0-5	主要功能缺失、代码无法运行、工作组织混乱、人工智能的使用被隐藏或文档不足。

14. 最低合格清单

提交前,学生应确认:

- 程序运行;
 - 至少有 7 门必修课;
 - 至少有 10 名玩家;
 - 至少有 15 位英雄;
 - 至少有 20 件设备;
 - 至少有 3 支队伍;
 - 至少有 10 条比赛记录;
 - 菜单系统运行正常;
 - 球员查询功能正常;
 - 团队概览工作;
 - 英雄细节工作;
- 设备统计工作;
 - 排行榜功能正常;
 - 管理员/玩家登录功能正常;
 - plan.md 文件存在且内容详细;
 - prompts.md包含实际的提示信息;
 - agent-log.md显示至少 3 个代理角色;
 - reflection.md回答了所有问题;
 - Git 至少有 12 次有意义的提交;
 - 已包含git-history.txt ;
 - 测试文档已包含在内。

附录 A:学生工作流程

- 1. 仔细阅读课程作业。
- 2. 创建一个 Git 仓库。
- 3. 在编写代码之前,先编写plan.md文件。
- 4. 向建筑师经纪人征求设计反馈意见。
- 5. 手动创建初始 Java 类。
- 6. 提交初始结构。
- 7. 仅对选定的方法使用实施代理,而不是一次性对整个项目使用。
- 8. 经常编译和运行。
- 9. 使用测试/审查代理来审查代码。
- 10. 修复漏洞并记录发生的情况。
- 11. 更新prompts.md、 agent-log.md和reflection.md。
- 12.导出Git历史记录。
- 13. 最后检查:确保每个类和方法都能解释清楚。

附录 B:快速质量指南

质量	弱提示	更强烈的提示
设计	“帮我编写Java项目。”	请为 Java 类设计提出建议 面向对象编程 《王者荣耀》信息 系统应包含继承、接口、集合和类职责。不要编写完整的代码。
实现方式	“编写这段代码。”	“仅实现球员查找功能” 使用现有播放器的方法， 下面列出英雄类和团队类。解释一下假设和特殊情况。
调试	“帮我修复代码。”	以下方法会崩溃 寻找一位无名英雄。 找出原因,解释原因,并 提出一个最小限度的解决方案。不要 重写不相关的代码。
审查	这样好吗？	“请检查此 Java 类的面向对象编程设计、封装、 集合使用以及潜在的空指针错误。 请提供具体意见。

附录 C:推荐的README.md结构

```
# 人工智能辅助的王者荣耀信息管理系统

## 1. 项目概述
```

2. 如何运行

3. 默认登录帐户

4. 已实现的功能

5. 使用的Java概念

6. 人工智能应用总结

7. 测试总结

8. 已知局限性

参考

1. 斯坦福大学 CS106B 课程存档。利用人工智能辅助编程。<https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1262/assignments/7-huffman-ai-coding/>
2. 斯坦福教学共享平台。制定人工智能课程政策。<https://teachingcommons.stanford.edu/teaching-guides/artificial-intelligence-teaching-guide/creating-your-course-policy-ai>
3. 斯坦福 SCALE 项目。 “Promptly:利用提示问题教授学习者如何有效地利用人工智能进行代码生成”。<https://scale.stanford.edu/ai/repository/promptly-using-prompt-problems-teach-learners-how-effectively-utilize-ai-code>